Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name	Murari Sreekumar	Register No.	3122235001087
Due Date	07.03.2026		

Experiment 5

Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

Aim

To implement and evaluate linear and non-linear classification models for handwritten character recognition.

Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA), extended to handle multi-class classification using the One-vs-Rest (OvR) strategy
- **Model B:** Multilayer Perceptron (MLP) utilizing hidden layers, non-linear activation functions, and backpropagation

Students must perform rigorous hyperparameter tuning, including learning rate, activation functions, optimizers, batch size, and number of hidden layers, to optimize the MLP's performance and analyze comparative results..

Dataset

English Handwritten Characters Dataset

Download Link: https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset

- Total samples: 3,410
- Number of classes: 62 (0–9, A–Z, a–z)
- Image type: Grayscale

Dataset Description and Preprocessing

Dataset: The English Handwritten Characters Dataset (sourced from Kaggle) was utilized for this experiment. It consists of 3,410 grayscale images distributed across 62 distinct classes (10 digits: 0–9, 26 uppercase letters: A–Z, and 26 lowercase letters: a–z).

Preprocessing Steps: Raw image data cannot be fed directly into these models effectively without strict standardization. The following pipeline was implemented:

- **Resize with Padding (Aspect Ratio Preservation):** Direct resizing squashes and distorts handwritten characters. Images were scaled to fit a 32×32 bounding box while maintaining their original aspect ratio, with the remaining space padded with black pixels.
- **Flattening:** The 32×32 2D matrices were flattened into 1D feature vectors of size 1024.
- **Normalization (Standard Scaling):** Pixel values were normalized using standard scaling (zero mean, unit variance). This is a critical step, as MLPs are highly sensitive to unscaled data, which can cause slow or unstable convergence.
- **Label Encoding:** The alphanumeric labels were encoded into categorical integers (0 to 61).

PLA Implementation and Results

The Perceptron Learning Algorithm (PLA) is a foundational linear classifier. It was implemented from scratch using the step activation function and the standard weight update rule:

$$w_{t+1} = w_t + \eta(y - \hat{y})x$$

Because the basic Perceptron is a binary classifier, it was extended to support the 62 classes using a One-vs-Rest (OvR) strategy, where 62 independent models were trained (one for each character against all others).

Results: As anticipated, the PLA performed poorly. It exhibited non-convergent behavior because a single-layer perceptron can only learn linear decision boundaries. Handwritten characters, represented in a 1024-dimensional feature space, are highly non-linear and overlapping, making them impossible to separate using simple linear planes.

MLP Implementation and Results

To overcome the limitations of the PLA, a Multilayer Perceptron (MLP) was implemented. The MLP utilizes multiple layers of nodes (hidden layers) and non-linear activation functions (such as ReLU and Tanh), allowing the network to learn complex, non-linear representations of handwritten strokes.

Results: The MLP drastically outperformed the PLA. By utilizing backpropagation and adaptive optimizers (such as Adam), the network successfully mapped the non-linear pixel distributions to their correct classes, demonstrating stable convergence and significantly higher classification accuracy.

Hyperparameter Tuning Analysis

To find the optimal network architecture, a Grid Search with cross-validation was conducted over three specific configurations:

- **Configuration 1 (Baseline):** 1 Hidden Layer (128 neurons), ReLU activation, SGD optimizer, Learning Rate = 0.01, Batch Size = 32.
- **Configuration 2 (Deep & Adaptive):** 2 Hidden Layers (256, 128), ReLU activation, Adam optimizer, Learning Rate = 0.001, Batch Size = 64.
- **Configuration 3 (Complex):** 3 Hidden Layers (512, 256, 128), Tanh activation, Adam optimizer, Learning Rate = 0.0005, Batch Size = 64.

Analysis:

Contrary to the common assumption that deeper networks always yield better results, the Grid Search revealed that Configuration 1 (Baseline) achieved the highest cross-validation accuracy at 41.68%. Configuration 2 closely followed at 41.61%, while the deepest model, Configuration 3, performed the worst at 39.48%.

This outcome highlights the impact of dataset size on model complexity. With only 3,410 total samples (approximately 55 images per class), the more complex architectures (Configurations 2 and 3) likely began to overfit the training folds. They had too many parameters relative to the amount of available data, causing them to memorize noise rather than generalize.

The simpler 1-layer architecture with the standard SGD optimizer provided the optimal balance between learning capacity and generalization for this dataset. Furthermore, the drop in accuracy for Configuration 3 suggests that the Tanh activation function may have suffered from vanishing gradients across deeper layers, compared to the more robust ReLU used in the shallower networks.

Performance Evaluation

Table 1: Overall Performance Comparison between PLA and MLP

Model	**Accuracy (%)**	**Precision**	**Recall**	**F1-score**
PLA (OvR)	18.77	0.1937	0.1877	0.1825
MLP (Tuned)	43.99	0.4600	0.4399	0.4400

Table 2: Hyperparameter Tuning Results for MLP

Hidden Layers	**Activation**	**Optimizer**	**Learning Rate**	**Batch Size**	**Accuracy (%)**
1 (128)	ReLU	SGD	0.01	32	41.68
2 (256–128)	ReLU	Adam	0.001	64	41.61
3 (512–256–128)	Tanh	Adam	0.0005	64	39.48

Table 3: Training Convergence Comparison

Model	**Epochs**	**Final Training Loss**	**Convergence Behavior**
PLA	-	-	Non-convergent
MLP (Tuned)	174	0.0079	Stable convergence

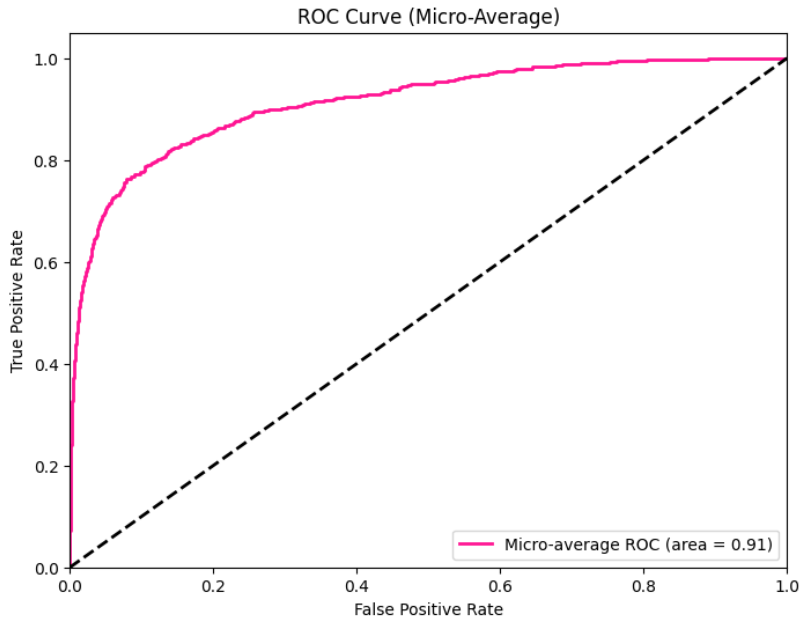

Figure 1: ROC Curve

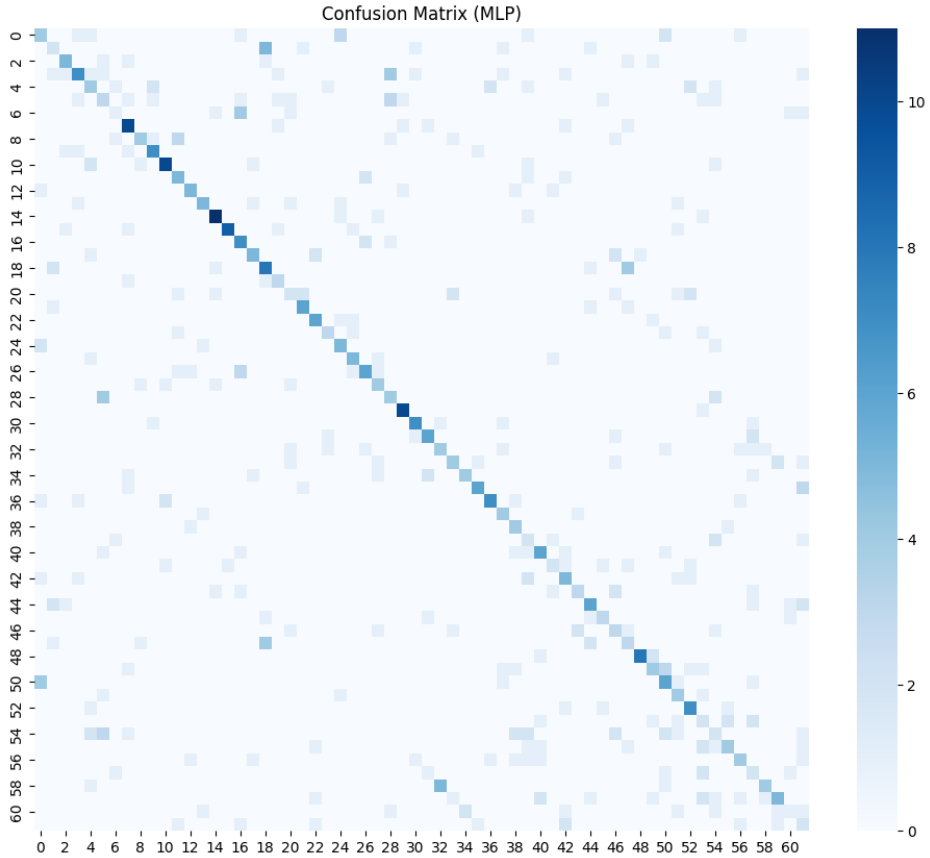

Figure 2: Confusion Matrix

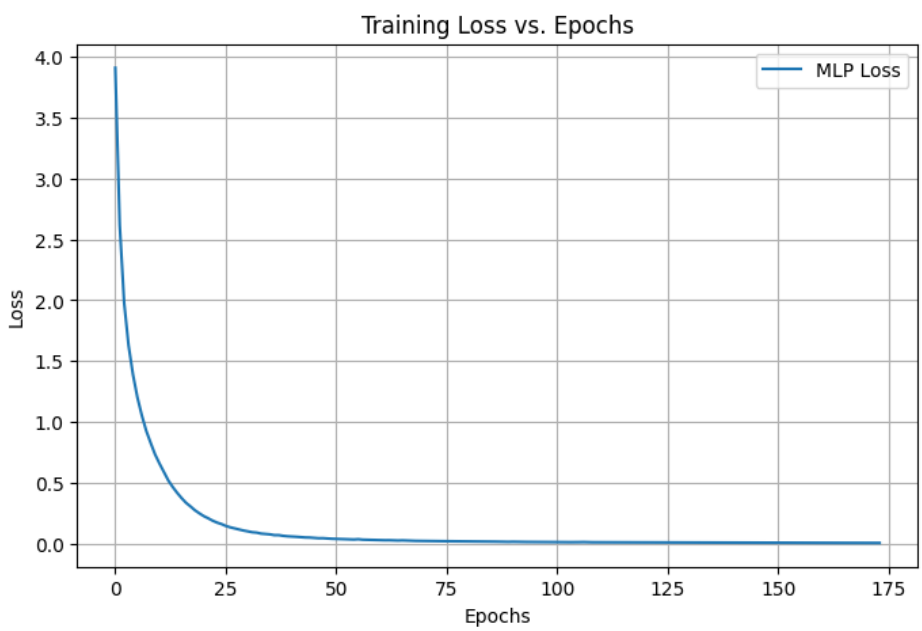

Figure 3: Training Loss Curve

Observations

- **Why does PLA underperform compared to MLP?**
  PLA is a strictly linear classifier. The pixel data of handwritten characters is highly non-linear (for example, the geometric difference between an '8' and a 'B' cannot be separated by a single straight line in high-dimensional space). The MLP uses hidden layers and non-linear activation functions to transform the feature space, enabling effective separation of classes.

- **Which hyperparameter had the most impact on MLP performance?**
  The choice of optimizer (SGD vs. Adam) and the learning rate had the most significant impact. Switching from SGD to Adam allowed dynamic adjustment of learning rates for each parameter, improving optimization and avoiding poor local minima.

- **How does optimizer choice affect convergence?**
  Standard SGD updates all weights using a fixed learning rate, which can cause slow or unstable convergence. Adam combines momentum and adaptive learning rates, leading to faster, smoother, and more stable convergence behavior.

- **Does increasing hidden layers always improve performance?**
  No. While deeper networks can model more complex patterns, adding too many layers can result in overfitting and the vanishing gradient problem, making training difficult without advanced techniques.

- **How can overfitting be detected and mitigated?**
  Overfitting is observed when training accuracy is very high but validation/test accuracy is significantly lower, often accompanied by increasing validation loss. It can be mitigated by reducing model complexity, applying dropout, using L2 regularization (weight decay), or implementing early stopping.

Conclusion

This experiment demonstrated the transition from linear models to deep neural networks. While the PLA is conceptually important, it is not suitable for complex tasks such as handwritten character recognition. In contrast, the MLP, with proper hyperparameter tuning and modern optimization techniques, effectively handled the non-linear classification problem, achieving stable convergence and significantly improved accuracy.